Alphafold protein database search engine

1 Introduction

The generation of protein structures using deep learning models is an exciting development in computational biology for researchers who study protein functions (Jumper et al. [2021]). The AlphaFold Protein Structure Database provides predicted structures for over 214 million proteins (Varadi et al. [2023]). This database is available as a Google BigQuery Public Dataset comprising 23 TB of predicted structures and 115 GB of metadata (Deepmind [2022]). We tackle the challenge of processing the 115 GB of metadata to develop a search engine for efficient access to the 23 TB of predicted protein structures. Our system follows an architecture based on a standard information retrieval pipeline: data collection, preprocessing, indexing, retrieval, and user interaction that we describe in this report. The system is deployed as a full-stack cloud application with the front-end hosted on Vercel and the back-end deployed on Google Cloud Platform. Our application is available at `https://protein-search-ttds.vercel.app`

2 Indexing

This section covers the preprocessing and indexing pre-computation that enables efficient search described in the next section. We implemented core functions in Rust with Python wrappers to integrate with the rest of the pipeline that was less compute, memory, and disk intensive. Rust was preferred over Python for the core for performance, and was integrated via the PyO3 Rust library.

2.1 Preprocessing

The AlphaFold Protein Structure Database has predicted protein structures and other metadata for 214,683,829 proteins for 1,007,288 organisms (Varadi et al. [2023]). For each protein, we process the following metadata: a numerical unique sequential protein ID ("Protein ID", "Doc ID") species-unique Taxonomy ID ("TaxID"), Species Name, Protein Name, Protein Description, Length of Amino Acid Sequence, and the Amino Acid Sequence (henceforth referred to as "sequence"). Each protein has a sequence of between 15 to 2700 amino acids, with an average length of 321 amino acids across all proteins.

The preprocessing stage converts raw protein sequence metadata spread over 450 files into a structured format for efficient indexing. Each protein is assigned a unique document identifier to ensure consistent indexing and lookup. The corresponding amino acid sequence is then split into n-grams, which serve as the vocabulary of terms for indexing. Since there are 20 amino acids, we have 20 unigrams, 400 bigrams, 8,000 trigrams, and 160,000 fourgrams. We explored the possibility of stop-word removal based on the frequency of unigram, but the most frequent unigram appears just twice as often as the rest of the unigrams, so we decided that using stop-word removal would complicate the application without sufficient benefit.

2.2 Index Representation

Indexing builds an inverted index over the amino acid n-grams (the TERMs) identified during preprocessing to map each TERM to proteins (DOC IDs), and POSITIONs (in the amino acid sequences). The `Ngram.starts, Ngram.offsets, Ngram.skips,` and `Ngram.data` files together form the inverted index for an n-gram and there are four sets of files depending on the length of the term: unigram, bigram, trigram, and fourgram. Indexing was implemented using a mix of Python and Rust. Indexing produces the following files:

1. `map_taxid.bin` (16 MB) stores the metadata file and protein ID ranges respective of each species, identified and sorted by their TaxID.

2. `map_name_to_taxid.csv` (35 MB) stores the species name respective of each TaxID, sorted alphabetically by species name.

3. `protein_offsets.bin` (2 MB): stores per-row byte lengths for `proteins.tsv`, enabling direct seeks to relevant rows without scanning the full file.

4. `species.tsv` (44 MB): stores species name, TaxID, and a mapping from protein ID to species to support constant-time retrieval of organism metadata once the protein ID is known.

5. `proteins.tsv` (10 GB) contains rows with 2 tab separated columns for the name and description of the protein. The name is used for retrieving the predicted structure.

6. `sequences.txt` (69 GB) contains amino acid sequences for all proteins.

7. `seq_offsets.bin` (2 GB) contains 8 bytes per protein for the offset of the amino acid sequence in `sequences.txt` to support quick access to the sequence length.

8. `Ngram.data` contains the postings of the TERMs as a vector of tuples (delta-encoded DOC ID, POSITION COUNT, variable-length delta-encoded vector of POSITIONs). The postings are decoded into fixed-width identifiers after reading from disk to enable efficient binary search and intersection operations. In spite of the compression, these files are large. The unigram, bigram, trigram, and fourgram files take 73 GB, 154 GB, 222 GB, and 264 GB respectively.

9. `Ngram.skips` contains a vector of tuples (DOC ID, DATA OFFSET) for a TERM. Since `Ngram.data` contains delta-encoded DOC IDs and variable length vectors for POSITIONs, for terms with a large number of documents (like all unigrams), a linear scan of the tuples in `Ngram.data` is slow. This scan is accelerated by the `Ngram.skips` file entry for the absolute DOC ID every 128 documents with the corresponding DATA OFFSET in the `Ngram.data` file. A binary search locates the range of at most 128 DOC IDs that need to be scanned linearly.

10. `Ngram.offsets` contains a vector of (DATA OFFSET, SKIP OFFSET, DOC COUNT, TERM COUNT) that holds a map from TERM to DATA OFFSET in `Ngram.data`, SKIP OFFSET in `Ngram.skips`, DOC COUNT specifying the number of proteins that contain this TERM, and TERM COUNT specifying the number of times this TERM appears in all proteins.

11. `Ngram.starts` contains a vector of (TERM, START) that holds a map from TERM to START in the `Ngrams.offsets` file. The TERMs are lexicographically sorted in ascending order to enable binary search.

To retrieve a result, the system performs a sequence of lookups across these files. Given a query term, binary search on `Ngram.starts` locates its entry index, which is used to read the corresponding `(DATA OFFSET, SKIP OFFSET, DOC COUNT)` tuple from `Ngram.offsets`. The skip structure in `Ngram.skips` accelerates traversal of the postings in `Ngram.data` to a target DOC ID. Once a matching protein ID is found, `seq_offsets.bin` provides the byte offset into `sequences.txt` to retrieve the amino acid sequence, while `protein_offsets.bin` enables a direct seek into `proteins.tsv` for the protein name and description. Organism metadata is resolved by looking up the protein ID in `species.tsv` and `proteins.tsv`. The `map_taxid.bin` and `map_name_to_taxid.csv` files are used to efficiently filter results by species, as explained in Section 3.4.

The index files as well as mapping files can be found here: Google Cloud Storage

2.3 Chunk-Based Processing

Indexing on commodity hardware required a chunk-based strategy. Groups of metadata files were processed independently to produce intermediate index files. Each chunk generated temporary `Ngram_M.starts`, `Ngram_M.offsets`, `Ngram_M.skips`, and `Ngram_M.data` files, where M denotes the chunk identifier. The chunk size was experimentally chosen to balance memory usage and disk capacity so that intermediate files could be processed on a system with limited RAM without excessive disk swapping. After all chunks were processed, the merge stage combined intermediate postings into a single global index using priority queues.

The chunk based processing allowed incremental indexing so that initially we processed just 1 metadata file for initial testing, then added a few more metadata files for more testing, and finally added the rest of the 450 files for indexing all 214 million proteins. The incremental indexing facility was critical for resuming the index build during development when the ideal chunk size was not known and the process exhausted RAM or disk space after processing a part of the metadata files.

2.4 Performance

The indexing pipeline was executed on two Google Cloud Compute virtual machines with an ARM64 architecture, 8 virtual CPUs, 64 GB of RAM, and 750 GB local NVMe SSD storage. The initial preprocessing took about 15 minutes, unigram indexing took 45 minutes, bigram took 1 hour, trigram took 1.5 hours, and fourgram took 2.5 hours. The unigram and bigram indexes were built on one machine while trigram and fourgram indexes are built on the other machine in parallel, and the entire process including copying the files to Google Cloud Storage required 4 hours.

3 Search Module

3.1 Query Language and Parsing

The query language has 20 case-insensitive letters for amino acids (`ACDEFGHIKLMNPQRSTVWY`), wildcards (`.` for single acid and `*` for zero or more acids), boolean operators (`&` for AND, `|` for OR, `~` for AND NOT), anchors (`^` for START and `$` for END) and parentheses for grouping.

The query interface permits filtering proteins by TaxID or species name and by length of the amino acid sequence and allows a limit on the number of retrieved results. Ranked search is offered as an option to rank results by TF-IDF score. Alternatively, an unranked ("normal") search can be performed, returning a random set of results that match the query and filters.

Queries are parsed using a recursive descent parser which constructs a parse tree representation. If the query consists solely of literal segments and anchors, it is executed directly. Otherwise, the tree is evaluated recursively, combining intermediate results according to the boolean operators.

Parsing decomposes long literal segments into n-grams using a greedy strategy that prefers longer n-grams. For example, a 10-character literal is decomposed into two fourgrams and one bigram. The segments are the terms that are looked up in the index.

3.2 Candidate Generation

For each n-gram, postings are retrieved from the index and converted into bitsets of protein IDs. Boolean operators are applied by combining these bitsets. The rarest n-grams are searched first as a heuristic to minimise intermediate bitset sizes.

Since the search language permits filtering proteins by TaxID or species name and by length of the amino acid sequence, these constraints are applied during this step. The protein IDs are grouped by species so the interval of contiguous IDs enables filtering the candidates by species.

3.3 Positional Verification

For candidate proteins, stored positions are checked to verify that n-grams appear in the correct order and distance. For complex queries, full sequence string matching with the appropriate gaps between segments is performed to ensure exact expression matching.

3.4 Filtering

Both species and length filters are applied to the documents during retrieval. While filtering by length is implemented as a simple comparison of the length of a document's sequence to the minimum and maximum allowed lengths, species filtering takes a two-step approach, with the first step being executed in Python before calling the Rust search function and the second during execution of the latter.

For this, a protein ID range corresponding to the species is obtained via binary search on the `map_taxid.bin` (TaxID → DocID range) and `map_name_to_taxid.csv` (Species Name → TaxID) files, which is then passed to the (Rust) search function as an argument pair. In the search function, all proteins with IDs outside the accepted range are then excluded from the results. This approach is possible due to the protein IDs being attributed after grouping proteins by species, therefore creating a 1:1:1 map from species name to TaxID to an interval of protein IDs.

All filters are, however, optional, and can be passed as Python `NoneType`s to the search function, which skips the filtering step if it is not needed.

3.5 Ranking

Ranking works best when the size of the result set is limited, so the search interface for ranked search requires a minimum of 5 amino acids in the query. For queries that return a large number of candidates, TF-IDF scores are computed for a million proteins before ranking the top candidates. For a query consisting of segments $1 \dots n$, the TF-IDF score of protein d is defined as:

$$Score_d = \frac{\sum_{i=1}^{n}(TF_{i,d} \times IDF_i)}{\ln(Length_d + 1)}.$$

$TF_{i,d}$ is the number of times segment i appears in protein d, $Length_d$ is the number of amino acids in protein d and

$$IDF_i = \ln\left(\frac{N_{total}}{N_{docs_with_i} + 1}\right)$$

where N_{total} is the total number of proteins and $N_{docs_with_i}$ is the number of proteins containing segment i. More complex models such as BM25 were considered but would introduce additional computational overhead without significant empirical benefit at this scale.

3.6 Protein Metadata Retrieval for Result Presentation

This component enables the search results page to display human-readable protein names and descriptions as well as the corresponding species that exhibits the protein, fetched respectively from the `proteins.tsv` and `species.tsv` files via binary search. The `protein_offsets.bin` file is used to skip over blocks of protein IDs to speed up this process. The protein name is also used when a result is subsequently clicked by the user to fetch additional information directly from the cloud, such as the 3D protein structure, exhibited in a page that provides extra organism context alongside structural protein visualization.

3.7 Efficiency

Search speed is primarily determined by the index structure. Memory mapping (`mmap`) is used extensively to load the 900GB index files efficiently into RAM, reducing system call overhead and disk I/O. Multi-core parallelism is used during bitmap construction for n-grams as well as sequence verification via string matching, and for queries with result limits, atomic counters coordinate across threads to terminate processing once sufficient results are found. An optimisation expands patterns such as `M.K` into 20 possible trigrams (e.g., MAK, MCK, ..., MYK), since performing multiple trigram lookups is faster than combining large postings sets for common acids.

4 Interface & User Experience

4.1 Front-end Architecture

The web interface was developed using ReactJS to provide intuitive interaction with the retrieval system which enables a modular and component-based design. The interface consists of three primary views:

- **Search Page:** This is the main landing page of the web interface, where the user can search for amino acid sequences using the query language and filters described in Section 3.1. Two search modes are available to the user: normal search, which returns an unordered set of results (with the search program terminating as soon as enough results have been returned) and ranked search, which returns the best scoring set of proteins that match the query, ranked according to the TF-IDF scores.
- **Search Results Page:** Displays the retrieved results that match the user query and displays them as a list containing the protein name, protein description, and species name. There is also information regarding the total number of matching proteins retrieved and the search time taken to retrieve the results. Each item in the list is clickable which takes the user to the corresponding protein structure page.
- **Protein Structure Page:** Provides detailed information and interactive 3D visualization of selected protein along with the users query sequence being highlighted within the protein sequence. The 3D visualization is rendered using JavaScript module name `3dmol` which is

specially designed to view molecular structures. The protein structure visualization dynamically renders structural data retrieved from cloud storage, allowing users to explore predicted AlphaFold structures interactively.

4.2 Back-end Services

Back-end services were implemented using Python's FastAPI Framework. The back-end supports the following two functions:

- Query processing and retrieval of protein metadata from the search index. The back-end service acts as an intermediary between the Rust search module and the web interface that the user interacts with.
- Fetching and parsing protein structure files from Google Cloud Storage for 3D rendering and populating the content in the protein structure page.

The RESTful API design enables efficient communication between the front-end and the search engine, ensuring low-latency response times even under concurrent access.

4.3 Front-end/Back-end Integration

The front-end and back-end are integrated through structured API endpoints. Search queries are sent to the back-end, which processes the query using the indexing and ranking pipeline before returning structured JSON responses.

For protein visualization, the back-end retrieves structural files from Google Cloud Public Storage and streams them to the front-end, enabling real-time rendering.

4.4 Deployment and Infrastructure

The system is deployed using a distributed cloud-based architecture:

- The React front-end is hosted on Vercel, providing global CDN distribution and streamlined deployment functionality using CI/CD workflows. The Vercel platform also provides a clean domain name which is a subdomain within the `vercel.app` domain
- Back-end services are deployed on a Google Cloud Platform (GCP) Virtual Machine. The virtual machine(c4a-highmem-16-lssd) being used is a high-memory (128 GB) with 1500 GB of Local SSD to support memory-mapped access of index files for search.

The deployment process includes provisioning the VM on GCP, copying the built index files from the Google bucket, installing required dependencies (Git, Python, Rust), compiling back-end services, and configuring secure communication between the Vercel front-end and the GCP VM containing the back-end service.

5 Evaluation

5.1 Normal Search Evaluation

Table 1 shows queries for search that we validated against the BigQuery Dataset (Deepmind [2022]). The initial queries test the query language and the last four queries with real amino acid sequences guided the optimizations of our search function. The last query is from an article on multiple sequence alignment of the ribosomal protein P0 (Wikipedia contributors [2026]).

Query	Retrieved Proteins	Time (s)
MVK	5,241,143	0.1
^M	198,702,062	8.7
LL$	1,577,602	6.2
M.K	67,367,281	3.6
^M.V	10,140,534	3.9
MVK & LL$	38,379	6.5
MVK | LL$	6,780,366	5.9
MVK ~ LL$	5,202,764	1.3
M*K	207,249,839	84.3
L*A*G*S*V	200,102,339	207.7
(MVK | LL$) & ^M	6,339,517	168.9
(M|L).(K|S)	195,920,841	110.9
AAAA*CCCC*EEEE*LLLL	10	0.183
TEPA*MTAD*DGRT	28	0.048
^MQIT*GCRR$	2	0.015
^MPREDRATWKSNYFLKIIQLLDDYPKCFIVGADNVGSKQMQQIRMSLRGKAVVLMGKNTMMRKAI RGHLENNPALEKLLPHIRGNVGFVFTKEDLTEIRDMLLANKVPAAARAGAIAPC.VTVPAQNTGLG PEKTSFFQALGITTKISRGTIEILSDVQLIKTGDKVGASEATLLNMLNISPFSFGL.IQQVFDNGS IYNPEVLDITEETLH.RFLEGVRNVASVCLQIGYPTVASVPHSIINGYKRVLA..VETDYTFPLAE KVKAFLADPSAFVAAAPVAAATTAAPAAAAAPAKVEAKEESEESDEDMGFGLFD$	31	0.501

Table 1: Query search number of matches and time in seconds for Rust search engine.

5.2 Ranked Search Evaluation Queries

Query	Proteins	Best Score	Worst Score	Time (s)
AAAA*CCCC*EEEE*LLLL	10	115.2	13.17	1.13
TEPA*MTAD*DGRT	28	2.645	2.350	1.16
^MQIT*GCRR$	2	2.347	2.347	1.00
^MPREDRATWKSNYFLKIIQLLDDYPKCFIVGADNVGSKQMQQIRMS LRGKAVVLMGKNTMMRKAIRGHLENNPALEKLLPHIRGNVGFVFTKE DLTEIRDMLLANKVPAAARAGAIAPC.VTVPAQNTGLGPEKTSFFQA LGITTKISRGTIEILSDVQLIKTGDKVGASEATLLNMLNISPFSFGL .IQQVFDNGSIYNPEVLDITEETLH.RFLEGVRNVASVCLQIGYPTV ASVPHSIINGYKRVLA..VETDYTFPLAEKVKAFLADPSAFVAAAPV AAATTAAPAAAAAPAKVEAKEESEESDEDMGFGLFD$	31	71.77	71.77	1.42

Table 2: Ranked search scores and time in seconds for Web interface.

Table 2 shows ranked search with TF-IDF scores of the first and last ranked proteins. The first query searches for common fourgram sequences that occur many times in proteins and finds 10 proteins where the top-ranked protein is long (1250 acids) but has many repetitions of A and E acids. The second query searches for relatively rare fourgram sequences that occur once in proteins and finds 28 proteins where the minor differences in the scores are due to the lengths of the sequences: the top-ranked protein has length 552 while the bottom-ranked protein has length 1222. The third and fourth queries look for rare fourgram sequences and return results with the same scores because all the matched proteins are of the same length.

For queries that match up to 1000 proteins, there is little difference between the time required for ranked search compared to normal search. However for queries that match millions of proteins,

the time required by ranked search is larger because it computes TF-IDF scores for up to a million matches before sorting them to return the top scoring proteins.

The increase in time for the queries in Table 2 compared to Table 1 is because the ranked search was executed through the Web interface that adds latency of about 1 second.

6 Discussion & Limitations

A key strength of our system lies in its index structure and search pipeline. The use of n-gram indexing combined with delta-encoded postings lists allows efficient storage and fast retrieval for a large collection. Additionally, the two-stage retrieval process ensures that candidate generation remains fast while positional verification guarantees correct sequence matching. Parallel processing and memory-mapped file access further improve performance by utilising hardware resources efficiently.

Despite these strengths, there are a few limitations. Wildcard queries containing multiple gaps (e.g., repeated use of $*$) can generate extremely large candidate sets, increasing query time. Although heuristic optimizations such as rare n-gram prioritization help mitigate this issue, query performance degrades for patterns that return large result sets.

Second, the evaluation primarily focuses on runtime performance and system correctness. Because the dataset lacks standard relevance judgements, metrics that depend on them, such as DCG and nDCG, could not be applied. Instead, the results were validated against known sequences and cross-checked with the Google BigQuery AlphaFold dataset.

Third, the index size remains large due to the scale of the dataset and the use of multiple n-gram levels. While compression techniques and memory mapping allow the system to operate efficiently on commodity hardware, further work could explore additional compression methods or distributed indexing strategies to reduce storage requirements.

Finally, this project is limited to retrieving proteins with amino acid sequences matching the textual query. It does not address other important questions such as finding the similarity between a set of proteins using measures such as BLASTP and finding proteins that match a given three dimensional structure.

7 Conclusion

This project presented the design and implementation of a scalable search engine for the AlphaFold Protein Structure Database. The core component is a retrieval system tailored to amino acid sequence search. Protein sequences are represented using multi-level n-gram inverted indices. The search pipeline combines efficient candidate generation with positional verification to ensure correct sequence matching while maintaining fast query performance, with most queries completing within seconds. A customised TF-IDF ranking model further enables ranked retrieval of protein sequences based on query relevance. The system also supports a flexible query language that allows wildcards, boolean expressions and multiple filters.

Beyond the retrieval engine, the system was deployed as a full-stack application. The React-based web interface allows users to submit queries, explore ranked results, and visualise protein structures interactively using 3D rendering. Back-end services implemented in FastAPI connect the interface with the Rust-based search engine and retrieve additional metadata and structural data from cloud storage.

In summary, the project demonstrates how a large biological dataset can be searched efficiently using modern information retrieval techniques.

References

Google Deepmind. AlphaFold Protein Structure Database, Dec 2022. URL https://github.com/google-deepmind/alphafold/blob/main/afdb/README.md.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596 (7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

Mihaly Varadi, Damian Bertoni, Paulyna Magana, Urmila Paramval, Ivanna Pidruchna, Malarvizhi Radhakrishnan, Maxim Tsenkov, Sreenath Nair, Milot Mirdita, Jingi Yeo, Oleg Kovalevskiy, Kathryn Tunyasuvunakool, Agata Laydon, Augustin Žídek, Hamish Tomlinson, Dhavanthi Hariharan, Josh Abrahamson, Tim Green, John Jumper, Ewan Birney, Martin Steinegger, Demis Hassabis, and Sameer Velankar. Alphafold protein structure database in 2024: providing structure coverage for over 214 million protein sequences. *Nucleic Acids Research*, 52(D1):D368–D375, Nov 2023. URL https://doi.org/10.1093/nar/gkad1011.

Wikipedia contributors. Multiple sequence alignment — Wikipedia, the free encyclopedia, 2026. URL https://en.wikipedia.org/w/index.php?title=Multiple_sequence_alignment&oldid=1335445064. [Online; accessed 4-March-2026].